

ECSE 420 Assignment 1

Feb 15 2026

Alisha Malik 261071602

Vivek Motta 260962299

Question 1.1

To implement matrix multiplication sequentially, the method `sequentialMultiplyMatrix(double[][] a, double[][] b)` uses the standard triple nested loop algorithm. The outer loop iterates over each row i of matrix a , the middle loop iterates over each column j of matrix b , and the inner loop iterates over the shared dimension k to compute the dot product between row i of a and column j of b . For each output entry, the method accumulates $a[i][k] * b[k][j]$ into a running sum and stores the final sum in `result[i][j]`. This matches the mathematical definition of matrix multiplication, where each result element depends only on one row of a and one column of b , and the resulting matrix has dimensions $rowsA \times colsB$.

To validate that the sequential method produces correct results, correctness is checked in multiple complementary ways. First, the output dimensions are implicitly validated by allocating `result` as new `double[rowsA][colsB]`, which ensures the method follows the expected multiplication rule $(m \times n) * (n \times p) = (m \times p)$. Second, the sequential output is compared against the parallel output produced on the same randomly generated inputs, using `validateResults(seqResult, parResult)` as an element by element oracle. Because floating point arithmetic can introduce tiny rounding differences, the validation uses a tolerance of $1e-6$, meaning two values are considered equal if their absolute difference is below that threshold. If validation passes for large random matrices and also for smaller sanity tests such as identity like cases or small hand checkable examples, this provides strong evidence the sequential implementation is correct.

Question 1.2

To implement matrix multiplication in parallel, the method `parallelMultiplyMatrix(double[][] a, double[][] b)` uses an `ExecutorService` created by `Executors.newFixedThreadPool()`. The computation is divided between threads using row based partitioning of the result matrix. The total number of rows `rowsA` is split into contiguous blocks, where `rowsPerThread = ceil(rowsA / NUMBER_THREADS)` determines how many rows each thread should compute. For each thread index t , the code computes `startRow = t * rowsPerThread` and `endRow = min(startRow + rowsPerThread, rowsA)`, then submits a task that computes all result entries for rows `startRow` through `endRow - 1`. Inside each task, the same dot product logic as the sequential method is applied, but only for the thread's assigned rows.

The results are combined implicitly by having every thread write directly into the shared result matrix. This works correctly without locks because each thread writes to a distinct set of rows, so no two threads ever write to the same `result[i][j]` location. Since there are no overlapping writes, there is no race condition on the result array, and each computed entry is deterministic given the inputs. The only required synchronization is ensuring that the main thread does not return the result matrix before all worker tasks finish. This is enforced by calling `executor.shutdown()` followed by `executor.awaitTermination(1, TimeUnit.HOURS)`, which blocks until all submitted tasks complete or the timeout occurs.

To validate that the parallel method works correctly, the parallel output is compared against the sequential output using `validateResults`. This method checks that matrix dimensions match and then compares each element with a tolerance of $1e-6$. If the validation consistently returns true across repeated runs and across different matrix sizes, it indicates the parallel decomposition did not miss any rows, did not compute any row twice, and did not introduce incorrect values due to unsafe shared writes. This validation approach is

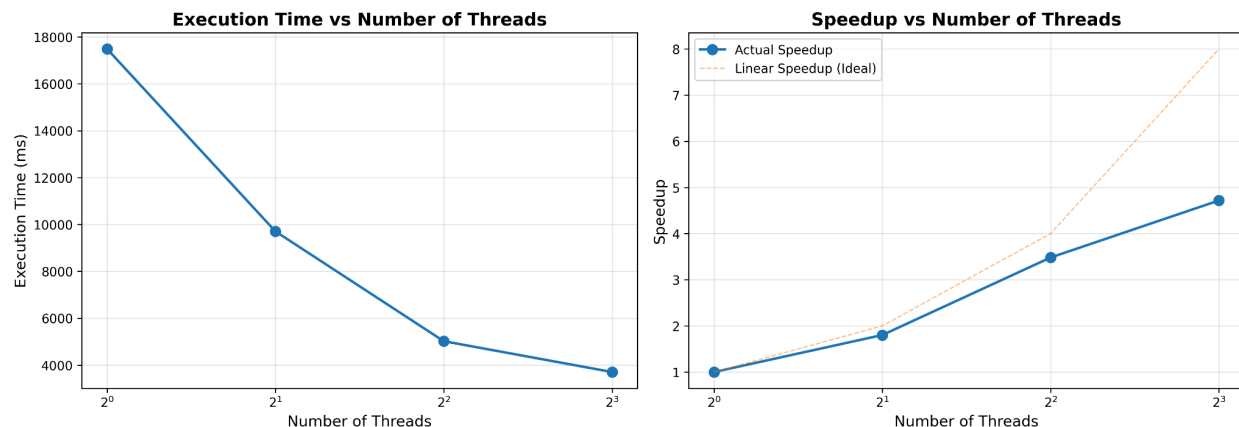
especially appropriate here because the sequential method is a straightforward reference implementation, so matching it element wise is a strong correctness guarantee.

Question 1.3

A timing method is added through `measureExecutionTime(double[][] a, double[][] b, boolean isParallel)`, which measures elapsed time using `System.nanoTime()` before and after calling either `sequentialMultiplyMatrix` or `parallelMultiplyMatrix`. The elapsed time is converted to milliseconds by dividing by `1_000_000.0`, which makes the results easier to interpret while still using a high resolution timer. In the provided main function, the same timing approach is used directly to measure sequential and parallel times, compute speedup as `seqTime / parTime`, and print the results. This gives a simple and consistent measurement framework for comparing performance.

To test that the timing method returns plausible results, the first check is repeatability. Running the same measurement multiple times should yield similar values, with some expected variation due to JVM warm up, garbage collection, and operating system scheduling. This is why the test programs average over multiple runs using `NUM_RUNS = 3`, which reduces the impact of outliers and produces more stable estimates. A second plausibility check is that speedup should generally be less than or approximately equal to the number of physical cores used effectively, because overhead and non parallelizable portions prevent perfect scaling. A third plausibility check is that for sufficiently large matrices, parallel time should be lower than sequential time, while for small matrices parallel time may be similar or worse due to thread pool overhead and task scheduling costs. Finally, plausibility also includes correctness coupling. Timing measurements are only meaningful if the produced results match, so it is important that `validateResults` returns true for the runs being benchmarked.

Question 1.4



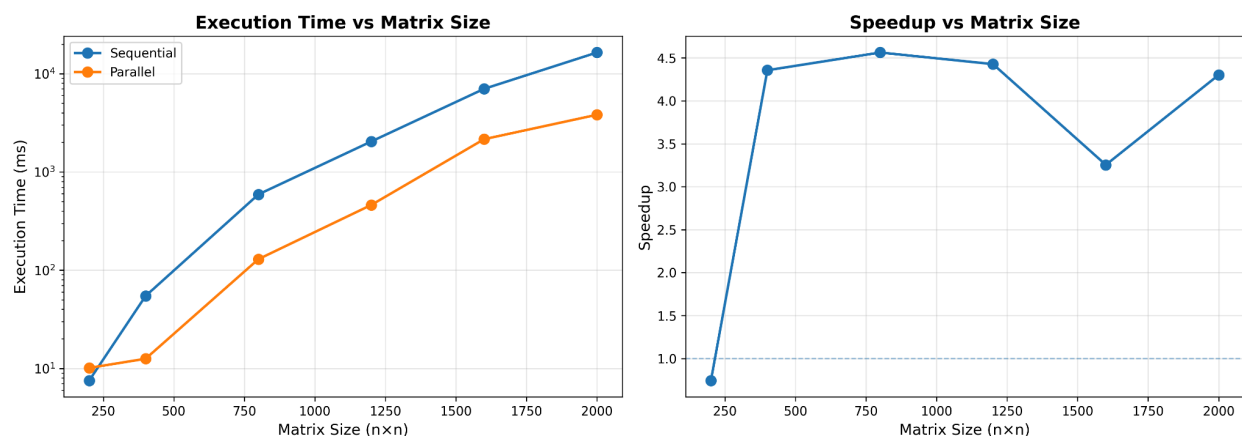
To study the effect of thread count on parallel matrix multiplication performance, the number of threads was varied while multiplying two fixed size 4000 by 4000 matrices. The execution time decreases substantially as the number of threads increases from one to four. Specifically, moving from one thread to two threads results in a large reduction in execution time, and increasing from two to four threads yields

another significant improvement. This indicates that the workload is effectively parallelized and that the available cores are being utilized efficiently in this range. The execution time continues to decrease when increasing the thread count from four to eight, but the improvement is smaller compared to earlier increases.

The speedup plot shows a similar trend. Speedup grows almost linearly at first, reaching approximately 1.8x with two threads and about 3.5x with four threads. When the number of threads is increased to eight, the speedup rises further to approximately 4.7x. However, this speedup is noticeably lower than the ideal linear speedup curve, which would predict an 8x improvement at eight threads. This divergence indicates that non parallelizable overhead and hardware limitations prevent perfect scaling.

The shape of both graphs demonstrates diminishing returns as the thread count increases. While adding threads initially provides strong benefits, the rate of improvement slows once the number of threads exceeds the effective parallel capacity of the system. This behavior can be attributed to several factors. First, the number of physical CPU cores limits the amount of true parallelism, so additional threads beyond that point compete for the same hardware resources. Second, thread management overhead, including task scheduling and context switching, becomes more significant as the thread count increases. Third, matrix multiplication is memory intensive, and multiple threads accessing large matrices simultaneously can saturate memory bandwidth and increase cache misses. As a result, the execution time curve flattens and the speedup curve bends downward relative to the ideal linear case.

Question 1.5



To examine how matrix size affects performance, both sequential and parallel matrix multiplication were executed for increasing matrix dimensions while using the optimal thread count identified in Question 1.4. The execution time for both methods increases rapidly as matrix size grows, reflecting the cubic time complexity of standard matrix multiplication. On the logarithmic execution time plot, both curves rise steeply with increasing matrix size, confirming that the dominant cost scales as $O(n^3)$.

For small matrices, particularly around 200 by 200, the parallel execution time is slightly worse than the sequential execution time, resulting in a speedup below 1. This occurs because the overhead of thread

creation, task submission, and synchronization outweighs the relatively small amount of computation required. As matrix size increases to 400 and 800, the parallel method becomes significantly faster than the sequential method, with speedup values rising sharply to above 4x. This indicates that the computational workload is now large enough to amortize parallel overhead and benefit from concurrent execution.

At larger matrix sizes, such as 1200 and 1600, the speedup remains high but shows some fluctuation. In particular, the speedup dips slightly around the 1600 by 1600 case before increasing again at 2000 by 2000. This variation is consistent with real system effects such as cache capacity limits, memory bandwidth contention, and JVM runtime behavior. As matrices grow larger, they no longer fit well in cache, increasing memory access costs and reducing parallel efficiency temporarily. Despite this dip, the parallel implementation consistently outperforms the sequential version for all large matrix sizes tested.

Overall, the speedup versus matrix size plot shows a characteristic pattern. Speedup is low or below one for very small matrices, increases rapidly once the workload becomes sufficiently large, and then stabilizes near a plateau determined by the number of threads and hardware limits. This behavior aligns with theoretical expectations from Amdahl's Law. For small problem sizes, fixed parallel overhead dominates total execution time. For large problem sizes, the parallelizable portion dominates, and speedup approaches an upper bound dictated by available cores and memory performance rather than continuing to grow indefinitely.

To run code:

```
javac -d ca/mcgill/ecse420/bin ca/mcgill/ecse420/a1/MatrixMultiplication.java
java -cp ca/mcgill/ecse420/bin ca.mcgill.ecse420.a1.MatrixMultiplication
```

Sample Code output:

Sequential execution time: 37215.731875 ms

Parallel execution time: 11009.638334 ms

Results match: true

Speedup: 3.3802865040598347x

Question 2.1

Deadlock is a situation in a concurrent system where two or more threads are permanently blocked because each thread is waiting for a resource held by another thread. Since none of the threads can proceed, the program stops making progress and may appear to hang indefinitely.

Deadlock can only occur when all four Coffman conditions are satisfied at the same time. The first condition is mutual exclusion, which means that a resource can only be held by one thread at a time. In the provided program, this condition is enforced by Java's synchronized blocks, which ensure that lockA and lockB can only be held by one thread at a time. The second condition is hold and wait, where a thread holds one resource while waiting to acquire another. This situation arises when a thread acquires one lock and then attempts to acquire a second lock. The third condition is no preemption, meaning that a lock cannot be forcibly taken from a thread and must be released voluntarily. Java's synchronization

mechanism satisfies this condition because a thread releases a lock only when it exits the synchronized block. The final condition is circular wait, where a cycle of threads exists such that each thread is waiting for a resource held by another thread in the cycle.

The provided program is designed to prevent deadlock by eliminating the circular wait condition. Both Thread1 and Thread2 acquire the shared locks in the same order, first lockA and then lockB. Because the lock acquisition order is consistent across threads, it is impossible for a circular dependency to form. Even though mutual exclusion, hold and wait, and no preemption still apply, deadlock cannot occur without circular wait. The Thread.sleep(100) calls increase the likelihood of contention and demonstrate that the program remains correct even under concurrent execution, as one thread will always complete its critical section before the other proceeds.

In general, deadlock can have serious consequences in software systems. A deadlocked program may become unresponsive, waste system resources by blocking threads indefinitely, and prevent other parts of the system from making progress. In many cases, the only way to recover from a deadlock is to restart the application, which can result in loss of work or service disruption.

Question 2.2

Deadlock can be avoided by breaking at least one of the four necessary conditions. One of the most common and effective techniques is resource ordering. This approach assigns a global order to all shared resources and requires that all threads acquire locks according to this order. By enforcing a consistent acquisition sequence, the circular wait condition is removed.

This strategy is directly applicable to the provided program and is already implemented. Both threads acquire lockA before attempting to acquire lockB, ensuring that no thread can hold lockB while waiting for lockA. As a result, at least one thread will always be able to acquire both locks, complete its work, and release the locks, allowing the other thread to proceed.

Other deadlock avoidance techniques include using lock timeouts, introducing lock hierarchies, using a single global lock, or redesigning the program to avoid holding multiple locks at the same time. While these approaches can also prevent deadlock, they often involve trade-offs such as increased complexity, reduced concurrency, or performance overhead.

For this program, resource ordering is the most appropriate solution because it is simple, efficient, and easy to reason about. It guarantees deadlock freedom while preserving concurrency, making it a practical and widely used approach in multi-threaded system design.

To run code:

```
javac -d ca/mcgill/ecse420/bin ca/mcgill/ecse420/a1/Deadlock.java
java -cp ca/mcgill/ecse420/bin ca.mcgill.ecse420.a1.Deadlock
```

Sample Code output:

DEADLOCK PREVENTION - LOCK ORDERING SOLUTION

Both threads acquire locks in the same order (A then B)

Thread 1: Lock A acquired

Thread 1: Attempting to acquire Lock B

Thread 1: Lock B acquired

Thread 1: Performing work with both locks

Thread 2: Lock A acquired

Thread 1: Released both locks

Thread 2: Attempting to acquire Lock B

Thread 2: Lock B acquired!

Thread 2: Performing work with both locks

Thread 2: Released both locks

SUCCESS! Both threads completed without deadlock.

Why this works:

- Both threads acquire locks in the same order (A -> B)
- No circular wait is possible
- One thread will complete before the other starts

Question 3.1

To ensure that two philosophers cannot hold the same chopstick simultaneously, each chopstick is implemented as a shared ReentrantLock object. A lock provides mutual exclusion because at most one thread can hold it at any instant. When a philosopher calls lock() on a chopstick that is already held by another philosopher, the calling thread blocks until the owner releases the lock. This enforces the requirement that a chopstick cannot be used by two philosophers at the same time. Internally, the lock records ownership and manages contention so that only the owning thread proceeds in the critical section, while other threads remain waiting until unlock() is called.

It is possible for philosophers to eventually become deadlocked in the naive dining philosophers design because all four Coffman conditions can hold simultaneously. Mutual exclusion holds because each chopstick is a lock held by only one philosopher. Hold and wait holds when a philosopher acquires one chopstick and then waits for the second. No preemption holds because a lock is not forcibly taken from a philosopher. Circular wait can arise if every philosopher acquires the same side first, creating a cycle of dependencies. In the classic deadlock execution, all philosophers become hungry at roughly the same time, each picks up their left chopstick, and then each blocks trying to acquire the right chopstick, which is held by their neighbor. Since nobody can acquire the second chopstick, nobody reaches the point where they can release their first chopstick, so the system makes no progress.

Sample output showing deadlock:

Philosopher 0 picked up first chopstick

Philosopher 1 picked up first chopstick

Philosopher 2 picked up first chopstick

Philosopher 3 picked up first chopstick

Philosopher 4 picked up first chopstick

[program hangs here - no further output]

Question 3.2

Deadlock is avoided in the submitted program using an asymmetric locking strategy that breaks the circular wait condition. Each philosopher tries to acquire two chopsticks in a fixed order. For philosophers 0 through $n-2$, the order is left then right. For the last philosopher $(n-1)$, the order is reversed, right then left. This single reversal prevents the system from reaching the state where every philosopher holds one chopstick while waiting for the other, because the dependency cycle cannot close consistently around the table. In a deadlock, each philosopher would need to be waiting on a chopstick held by their neighbor in a perfect ring. However, since the last philosopher waits in the opposite direction, the necessary cycle of “holds first, waits for second” does not form for all philosophers simultaneously. As a result, at least one philosopher is able to acquire both chopsticks, eat, and release them, which allows others to proceed.

Starvation refers to the situation where a particular philosopher is delayed indefinitely even though the system as a whole continues making progress. Starvation is possible in some deadlock-free solutions when lock acquisition is unfair, since the scheduler may repeatedly favor some threads. In this program, the chopsticks are constructed with `new ReentrantLock(true)`, which enables fairness. A fair `ReentrantLock` attempts to grant the lock to the longest waiting thread, rather than allowing newly arrived threads to repeatedly barge ahead. This significantly reduces the likelihood of starvation in practice, and it provides a stronger fairness argument than the default unfair lock. That said, starvation freedom for the dining philosophers problem is a global property involving two locks per eating attempt. Fairness is enforced per chopstick, not as a single combined “two chopsticks at once” allocation rule. In other words, fairness on each lock supports progress for waiting philosophers, but it does not, by itself, create a strict mathematical guarantee that every philosopher will eat within a bounded time under all possible schedules.

If a stronger starvation freedom guarantee is required, the program can be extended with an additional fairness mechanism at a higher level than individual chopstick locks, such as a fair waiter (for example, a fair semaphore limiting the number of philosophers who may attempt to pick up chopsticks concurrently) or a centralized queue that grants eating permission in FIFO order. These approaches ensure bounded waiting at the level of permission to try eating, not only at the level of acquiring a single chopstick.

To test for starvation freedom empirically, the implementation can record `timesEaten` for each philosopher and run the simulation for a long duration or until a large total number of eating events occurs. A practical test is to check that every philosopher’s counter continues increasing over time and that the minimum counter does not remain near zero while others grow. Additional checks can include comparing the minimum and maximum counts after long runs and repeating experiments across multiple runs to reduce dependence on a single random schedule. If any philosopher’s counter stops increasing for an extended period while others continue, that is evidence of starvation or extreme unfairness in the schedule.

To run code:

```
javac -d ca/mcgill/ecse420/bin ca/mcgill/ecse420/a1/DiningPhilosophers.java
java -cp ca/mcgill/ecse420/bin ca.mcgill.ecse420.a1.DiningPhilosophers
```


Sample output showing successful deadlock avoidance and starvation prevention:

Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 2 picked up first chopstick
Philosopher 2 picked up second chopstick
Philosopher 2 is eating
Philosopher 0 picked up first chopstick
Philosopher 0 picked up second chopstick
Philosopher 0 is eating
Philosopher 2 finished eating (total: 1 times)
Philosopher 3 picked up first chopstick
Philosopher 2 put down chopsticks
Philosopher 2 is thinking
Philosopher 3 picked up second chopstick
Philosopher 3 is eating
...
[output continues with all philosophers eating repeatedly]
...
Philosopher 1 finished eating (total: 156 times)
Philosopher 0 finished eating (total: 135 times)
Philosopher 2 finished eating (total: 158 times)
Philosopher 3 finished eating (total: 175 times)
Philosopher 4 finished eating (total: 135 times)
Final eat counts after 30 seconds:
Philosopher 0: 135 times
Philosopher 1: 156 times
Philosopher 2: 158 times
Philosopher 3: 175 times
Philosopher 4: 135 times

*All philosophers ate continuously with balanced counts, demonstrating no deadlock and no starvation.
The program ran successfully until manually stopped (exit code 130)*

Question 4.1

We are given:

Sequential portion: 30% ($s = 0.3$)

Parallel portion: 70% ($p = 0.7$)

n processors

Total execution time on single processor: 1 unit

Amdahl's Law:

The speedup formula is: $\text{Speedup} = \frac{1}{s + \frac{p}{n}}$

Where:

s = fraction of sequential execution time

p = fraction of parallelizable execution time

n = number of processors

Sequential time: 0.3

Parallel time on n processors: $\frac{0.7}{n}$

Derive the speedup formula $\text{Speedup}(n) = \frac{T_1}{T_n}$

$$\text{Speedup}(n) = \frac{1}{0.3 + \frac{0.7}{n}}$$

$$\text{Speedup}(n) = \frac{n}{0.3n + 0.7}$$

Find maximum speedup (as $n \rightarrow \infty$) When n approaches infinity, $\frac{0.7}{n}$ approaches 0

$$\lim_{n \rightarrow \infty} \frac{0.7}{n} = 0$$

Therefore Maximum Speedup:

$$\text{Speedup}_{\max} = \frac{1}{0.3} = \frac{10}{3} \approx 3.33$$

Question 4.2

We are given:

Original sequential portion: 40% ($s = 0.4$)

Original speedup on n processors: s_n

Improved sequential time decreased by factor k

double the original speedup $s'_n = 2s_n$

Original speedup

$$s_n = \frac{1}{0.4 + \frac{0.6}{n}}$$

Improved speedup

$$s'_n = \frac{1}{\frac{0.4}{k} + \frac{1 - \frac{0.4}{k}}{n}}$$

After reducing sequential time by factor k :

New sequential portion: $0.4/k$

New parallel portion: $1 - 0.4/k$

$$s'_n = 2s_n$$

$$\frac{1}{\frac{0.4}{k} + \frac{1 - \frac{0.4}{k}}{n}} = \frac{2}{0.4 + \frac{0.6}{n}}$$

Solve for k

Cross-multiplying:

$$0.4 + \frac{0.6}{n} = 2 \left(\frac{0.4}{k} + \frac{1 - \frac{0.4}{k}}{n} \right)$$
$$0.4 + \frac{0.6}{n} = \frac{0.8}{k} + \frac{2(1 - \frac{0.4}{k})}{n}$$

Multiply through by n :

$$0.4n + 0.6 = \frac{0.8n}{k} + 2 \left(1 - \frac{0.4}{k} \right)$$

$$0.4n + 0.6 = \frac{0.8n}{k} + 2 - \frac{0.8}{k}$$

Rearranging:

$$0.4n - 2 + 0.6 = \frac{0.8n}{k} - \frac{0.8}{k}$$

$$0.4n - 1.4 = \frac{0.8n - 0.8}{k}$$

$$k = \frac{0.8n - 0.8}{0.4n - 1.4}$$

Simplifying:

$$k = \frac{0.8(n - 1)}{0.4(n - 3.5)}$$

$$k = \frac{2(n - 1)}{n - 3.5}$$

$$k = \frac{4(n - 1)}{2n - 7}$$

Question 4.3

We are given:

Sequential time reduced 3-fold (divided by 3)

Modified program takes half the time on n processors

We need to find original sequential fraction

Original time:

$$T_1 = s + \frac{1-s}{n}$$

Modified time:

$$T'_1 = \frac{s}{3} + \frac{1 - \frac{s}{3}}{n}$$

Apply condition

$$T'_1 = \frac{T_1}{2}$$

$$\frac{s}{3} + \frac{1 - \frac{s}{3}}{n} = \frac{1}{2} \left(s + \frac{1-s}{n} \right)$$

Solve for s

Multiply by 6n:

$$2sn + 6 \left(1 - \frac{s}{3} \right) = 3sn + 3(1-s)$$

$$2sn + 6 - 2s = 3sn + 3 - 3s$$

$$6 - 3 = 3sn - 2sn - 3s + 2s$$

$$3 = sn - s$$

$$3 = s(n - 1)$$

$$s = \frac{3}{n-1}$$

This represents what fraction of the overall execution time the sequential part originally accounted for, as a function of n processors.

Appendix: Source Code

Question 1 Code:

```
1 package ca.mcgill.ecse420.a1;
2
3 import java.util.concurrent.ExecutorService;
4 import java.util.concurrent.Executors;
5 import java.util.concurrent.TimeUnit;
6
7 public class MatrixMultiplication {
8
9     private static final int NUMBER_THREADS = 4;
10    private static final int MATRIX_SIZE = 2000;
11
12    public static void main(String[] args) {
13
14        // Generate two random matrices, same size
15        double[][] a = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);
16        double[][] b = generateRandomMatrix(MATRIX_SIZE, MATRIX_SIZE);
17
18        // Test and time sequential multiplication
19        long startTime = System.nanoTime();
20        double[][] seqResult = sequentialMultiplyMatrix(a, b);
21        long endTime = System.nanoTime();
22        double seqTime = (endTime - startTime) / 1_000_000.0; // Convert to milliseconds
23        System.out.println("Sequential execution time: " + seqTime + " ms");
24
25        // Test and time parallel multiplication
26        startTime = System.nanoTime();
27        double[][] parResult = parallelMultiplyMatrix(a, b);
28        endTime = System.nanoTime();
29        double parTime = (endTime - startTime) / 1_000_000.0;
30        System.out.println("Parallel execution time: " + parTime + " ms");
31
32        // Validate that both methods produce the same results
33        boolean isValid = validateResults(seqResult, parResult);
34        System.out.println("Results match: " + isValid);
35
36        // Calculate speedup
37        double speedup = seqTime / parTime;
38        System.out.println("Speedup: " + speedup + "x");
39    }
40}
```

```

41  /**
42   * Returns the result of a sequential matrix multiplication
43   * The two matrices are randomly generated
44   * @param a is the first matrix
45   * @param b is the second matrix
46   * @return the result of the multiplication
47   */
48  public static double[][] sequentialMultiplyMatrix(double[][] a, double[][] b) {
49      int rowsA = a.length;
50      int colsA = a[0].length;
51      int colsB = b[0].length;
52
53      double[][] result = new double[rowsA][colsB];
54
55      // Standard matrix multiplication: C[i][j] = sum of A[i][k] * B[k][j]
56      for (int i = 0; i < rowsA; i++) {
57          for (int j = 0; j < colsB; j++) {
58              double sum = 0.0;
59              for (int k = 0; k < colsA; k++) {
60                  sum += a[i][k] * b[k][j];
61              }
62              result[i][j] = sum;
63          }
64      }
65
66      return result;
67  }
68
69  /**
70   * Returns the result of a concurrent matrix multiplication
71   * The two matrices are randomly generated
72   * @param a is the first matrix
73   * @param b is the second matrix
74   * @return the result of the multiplication
75   */
76  public static double[][] parallelMultiplyMatrix(double[][] a, double[][] b) {
77      int rowsA = a.length;
78      int colsA = a[0].length;
79      int colsB = b[0].length;
80

```

```

81      double[][] result = new double[rowsA][colsB];
82
83      // Create thread pool
84      ExecutorService executor = Executors.newFixedThreadPool(NUMBER_THREADS);
85
86      // Divide work by rows - each task computes one or more complete rows
87      int rowsPerThread = (int) Math.ceil((double) rowsA / NUMBER_THREADS);
88
89      for (int t = 0; t < NUMBER_THREADS; t++) {
90          final int startRow = t * rowsPerThread;
91          final int endRow = Math.min(startRow + rowsPerThread, rowsA);
92
93          executor.submit(() -> {
94              // Each thread computes rows from startRow to endRow
95              for (int i = startRow; i < endRow; i++) {
96                  for (int j = 0; j < colsB; j++) {
97                      double sum = 0.0;
98                      for (int k = 0; k < colsA; k++) {
99                          sum += a[i][k] * b[k][j];
100                      }
101                      result[i][j] = sum;
102                  }
103              }
104          });
105      }
106
107      // Shutdown executor and wait for all tasks to complete
108      executor.shutdown();
109      try {
110          executor.awaitTermination(1, TimeUnit.HOURS);
111      } catch (InterruptedException e) {
112          e.printStackTrace();
113      }
114
115      return result;
116  }
117

```

```

120  * @param a first matrix
121  * @param b second matrix
122  * @return true if matrices are equal, false otherwise
123  */
124  public static boolean validateResults(double[][] a, double[][] b) {
125      if (a.length != b.length || a[0].length != b[0].length) {
126          return false;
127      }
128
129      double tolerance = 1e-6; // Account for floating point precision errors
130
131      for (int i = 0; i < a.length; i++) {
132          for (int j = 0; j < a[0].length; j++) {
133              if (Math.abs(a[i][j] - b[i][j]) > tolerance) {
134                  return false;
135              }
136          }
137      }
138
139      return true;
140  }
141
142  /**
143   * Measures execution time for matrix multiplication
144   * @param a first matrix
145   * @param b second matrix
146   * @param isParallel whether to use parallel or sequential method
147   * @return execution time in milliseconds
148   */
149  public static double measureExecutionTime(double[][] a, double[][] b, boolean isParallel) {
150      long startTime = System.nanoTime();
151
152      if (isParallel) {
153          parallelMultiplyMatrix(a, b);
154      } else {
155          sequentialMultiplyMatrix(a, b);
156      }
157
158      long endTime = System.nanoTime();

```

```

149  public static double measureExecutionTime(double[][] a, double[][] b,
150      return (endTime - startTime) / 1_000_000.0; // Convert to milliseconds
151  }
152
153  /**
154   * Populates a matrix of given size with randomly generated integers between 0-10.
155   * @param numRows number of rows
156   * @param numCols number of cols
157   * @return matrix
158   */
159  public static double[][] generateRandomMatrix(int numRows, int numCols) {
160      double[][] matrix = new double[numRows][numCols];
161      for (int row = 0; row < numRows; row++) {
162          for (int col = 0; col < numCols; col++) {
163              matrix[row][col] = (int) (Math.random() * 10.0);
164          }
165      }
166      return matrix;
167  }
168
169  }

```


Question 2 Code:

```
1 package ca.mcgill.ecse420.a1;
2
3 /**
4  * Deadlock Using Lock Ordering
5  * This program demonstrates how to prevent deadlock by ensuring all threads
6  * acquire locks in the same order.
7  */
8 public class Deadlock {
9
10     private static final Object lockA = new Object();
11     private static final Object lockB = new Object();
12
13     /**
14      * Thread 1 acquires locks in order: A then B
15      */
16     static class Thread1 extends Thread {
17         @Override
18         public void run() {
19             // Acquire locks in consistent order: A then B
20             synchronized (lockA) {
21                 System.out.println("Thread 1: Lock A acquired");
22
23                 try {
24                     Thread.sleep(100);
25                 } catch (InterruptedException e) {
26                     e.printStackTrace();
27                 }
28
29                 System.out.println("Thread 1: Attempting to acquire Lock B");
30
31                 synchronized (lockB) {
32                     System.out.println("Thread 1: Lock B acquired");
33                     System.out.println("Thread 1: Performing work with both locks");
34                 }
35             }
36             System.out.println("Thread 1: Released both locks");
37         }
38     }
39 }
```

```
40
41 /**
42  * Thread 2 ALSO acquires locks in order: A then B
43  * This consistent ordering prevents deadlock
44  */
45 static class Thread2 extends Thread {
46     @Override
47     public void run() {
48         // Acquire locks in SAME order as Thread 1: A then B
49         synchronized (lockA) {
50             System.out.println("Thread 2: Lock A acquired");
51
52             try {
53                 Thread.sleep(100);
54             } catch (InterruptedException e) {
55                 e.printStackTrace();
56             }
57
58             System.out.println("Thread 2: Attempting to acquire Lock B");
59
60             synchronized (lockB) {
61                 System.out.println("Thread 2: Lock B acquired!");
62                 System.out.println("Thread 2: Performing work with both locks");
63             }
64             System.out.println("Thread 2: Released both locks");
65         }
66     }
67
68     public static void main(String[] args) {
69         System.out.println("DEADLOCK PREVENTION - LOCK ORDERING SOLUTION");
70         System.out.println("\nBoth threads acquire locks in the same order (A then B)\n");
71
72         Thread1 t1 = new Thread1();
73         Thread2 t2 = new Thread2();
74
75         t1.start();
76         t2.start();
77     }
78 }
```

```

78         try {
79             t1.join();
80             t2.join();
81         } catch (InterruptedException e) {
82             e.printStackTrace();
83         }
84
85         System.out.println("SUCCESS! Both threads completed without deadlock.");
86         System.out.println("\nWhy this works:");
87         System.out.println("- Both threads acquire locks in the same order (A -> B)");
88         System.out.println("- No circular wait is possible");
89         System.out.println("- One thread will complete before the other starts");
90     }
91 }
92

```

Question 3 Code:

3.1

```

J DiningPhilosophers deadlock.java
1  package ca.mcgill.ecse420.a1;
2
3  import java.util.concurrent.locks.ReentrantLock;
4
5  /**
6   * Dining Philosophers problem for Q3.1.
7   * This version does NOT avoid deadlock on purpose.
8   * All philosophers pick up the left chopstick first, then the right.
9   * This can eventually lead to the classic circular-wait deadlock.
10  */
11  public class DiningPhilosophers {
12
13      public static void main(String[] args) {
14          int numberOfPhilosophers = 5; // should work for any n >= 2
15          Philosopher[] philosophers = new Philosopher[numberOfPhilosophers];
16          ReentrantLock[] chopsticks = new ReentrantLock[numberOfPhilosophers];
17
18          // Chopsticks are shared objects (locks) so two philosophers cannot hold the same chopstick.
19          // Use default (unfair) or fair, deadlock is still possible either way.
20          for (int i = 0; i < numberOfPhilosophers; i++) {
21              chopsticks[i] = new ReentrantLock(); // intentionally not solving starvation/deadlock here
22              // chopsticks[i] = new ReentrantLock(true); // also fine, still can deadlock
23          }
24
25          // Everyone uses the SAME acquisition order: left then right.
26          // This makes deadlock possible.
27          for (int i = 0; i < numberOfPhilosophers; i++) {
28              ReentrantLock left = chopsticks[i];
29              ReentrantLock right = chopsticks[(i + 1) % numberOfPhilosophers];
30              philosophers[i] = new Philosopher(i, left, right);
31          }
32
33          for (int i = 0; i < numberOfPhilosophers; i++) {
34              new Thread(philosophers[i], "Philosopher-" + i).start();
35          }
36      }
37  }

```

```

37
38 public static class Philosopher implements Runnable {
39     private final int id;
40     private final ReentrantLock leftChopstick;
41     private final ReentrantLock rightChopstick;
42     private int timesEaten = 0;
43
44     public Philosopher(int id, ReentrantLock leftChopstick, ReentrantLock rightChopstick) {
45         this.id = id;
46         this.leftChopstick = leftChopstick;
47         this.rightChopstick = rightChopstick;
48     }
49
50     @Override
51     public void run() {
52         try {
53             while (true) {
54                 think();
55
56                 // Intentionally deadlock-prone order:
57                 // pick up left first, then right.
58                 leftChopstick.lock();
59                 System.out.println("Philosopher " + id + " picked up LEFT");
60
61                 // Small delay makes the deadlock scenario show up faster in practice.
62                 Thread.sleep(1);
63
64                 rightChopstick.lock();
65                 System.out.println("Philosopher " + id + " picked up RIGHT");
66
67                 eat();
68                 timesEaten++;
69                 System.out.println("Philosopher " + id + " finished eating (total: " + timesEaten + ")");
70
71                 rightChopstick.unlock();
72                 leftChopstick.unlock();
73                 System.out.println("Philosopher " + id + " put down chopsticks");
74             }

```

```

75         } catch (InterruptedException e) {
76             Thread.currentThread().interrupt();
77         }
78     }
79
80     private void think() throws InterruptedException {
81         System.out.println("Philosopher " + id + " is thinking");
82         Thread.sleep((long) (Math.random() * 150));
83     }
84
85     private void eat() throws InterruptedException {
86         System.out.println("Philosopher " + id + " is eating");
87         Thread.sleep((long) (Math.random() * 150));
88     }
89 }
90
91

```

3.2

```

package ca.mcgill.ecse420.a1;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Dining Philosophers problem - prevents deadlock using asymmetric ordering.
 */

public class DiningPhilosophers {

```

```

/**
 * Main method to run the dining philosophers simulation.
 *
 * @param args command line arguments
 */
public static void main(String[] args) {
    int numberOfPhilosophers = 5;
    Philosopher[] philosophers = new Philosopher[numberOfPhilosophers];
    ReentrantLock[] chopsticks = new ReentrantLock[numberOfPhilosophers];
    // initialize the chopsticks as locks
    for (int i = 0; i < numberOfPhilosophers; i++) {
        chopsticks[i] = new ReentrantLock(true);
    }
    //this is creating philosopher threads
    for (int i = 0; i < numberOfPhilosophers; i++) {
        ReentrantLock leftChopstick = chopsticks[i];
        ReentrantLock rightChopstick = chopsticks[(i + 1) %
numberOfPhilosophers];
        //last philosopher picks up right first then left to break circular
wait
        if (i == numberOfPhilosophers - 1) {
            philosophers[i] = new Philosopher(i, rightChopstick,
leftChopstick);
        } else {
            philosophers[i] = new Philosopher(i, leftChopstick,
rightChopstick);
        }
    }
    //starting all the threads
    for (int i = 0; i < numberOfPhilosophers; i++) {
        Thread t = new Thread(philosophers[i]);
        t.start();
    }
}
/**
 * Represents a philosopher who alternates between thinking and eating.
 */
public static class Philosopher implements Runnable {
    private int philosopherId;
    private ReentrantLock firstChopstick;
    private ReentrantLock secondChopstick;
}

```

```

private int timesEaten;
/**
 * Creates a new Philosopher.
 *
 * @param id philosopher identifier
 * @param first first chopstick to acquire
 * @param second second chopstick to acquire
 */
public Philosopher(int id, ReentrantLock first, ReentrantLock second) {
    this.philosopherId = id;
    this.firstChopstick = first;
    this.secondChopstick = second;
    this.timesEaten = 0;
}
/**
 * Executes the philosopher's think-eat cycle.
 */
@Override
public void run() {
    try {
        while (true) {
            // thinking phase
            think();
            //pick up first chopstick
            firstChopstick.lock();
            System.out.println("Philosopher " + philosopherId + "
picked up first chopstick");
            //pick up second chopstick
            secondChopstick.lock();
            System.out.println("Philosopher " + philosopherId + "
picked up second chopstick");
            //here is the eating phase
            eat();
            timesEaten = timesEaten + 1;
            System.out.println("Philosopher " + philosopherId + "
finished eating (total: " + timesEaten + " times)");
            //release second chopstick
            secondChopstick.unlock();
            //release first chopstick
            firstChopstick.unlock();
        }
    }
}

```

```

        System.out.println("Philosopher " + philosopherId + " put
down chopsticks");
    }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
/**
 * This is the function for thinking.
 *
 * @throws InterruptedException if interrupted
 */
private void think() throws InterruptedException {
    System.out.println("Philosopher " + philosopherId + " is
thinking");
    Thread.sleep((long) (Math.random() * 150));
}
/**
 * This is the function for eating.
 *
 * @throws InterruptedException if interrupted
 */
private void eat() throws InterruptedException {
    System.out.println("Philosopher " + philosopherId + " is eating");
    Thread.sleep((long) (Math.random() * 150));
}
}
}

```